

4. Change to the directory *contacts*. Check if the set-up is proper, by running the basic application:

```
npm start
```

This will start the application in development mode and 'Welcome to React' will be displayed.

5. Install the modules for Redux integration:

```
npm install -S redux
npm install -S redux-thunk
npm install -S react-redux
```

6. Install the modules, *toastr* and *jquery*, to display status on the browser when performing UI actions.

```
npm install -S toastr
npm install -S jquery
```

7. Install the HTTP client module, *axios*, to get data from the server.

```
npm install -S axios
```

8. Install the *express* and *body-parser* modules to serve the persistent data required for the Contacts application.

```
npm install -S express
npm install -S body-parser
```

9. To proxy API requests during development and avoid CORS issues, update *package.json* present in the *contacts* folder with the proxy setting (assuming that the Express server runs at port 4000).

```
"proxy": "http://localhost:4000"
```

Now we can start working on adding the features required for the Contacts application. While explaining the implementation, main code snippets have been provided. For the complete code, please refer to the GitHub repository at <https://github.com/srini-wip/react-redux-contacts.git>.

Serving the data required by the Contacts application using Express

We will now implement the code to serve the data required by the Contacts application using the Express Web framework.

Create a file *server.js* under the folder *contacts*. To provide the list of contacts, we will implement the API endpoint */api/contacts* using the GET method. This will read the JSON file, *contacts.json*, for the list of available contacts and send the response as JSON data.

```
app.get('/api/contacts', function(req, res) {
  // read from JSON file and send the response as JSON data
});
```

To add a contact to the contact list, we implement the API endpoint *api/contacts* using the POST method. To retrieve the data sent by the browser, we use the *body-parser* middleware, which inserts the required data in the request object, from which we can easily extract and save it to the JSON file.

```
app.post('/api/contacts', function(req, res) {
  // extract the data and save to JSON file
});
```

To delete a contact from the contact list, we implement the API endpoint *api/contacts/:id* using the DELETE method. This will delete the contact and update the JSON file.

```
app.delete('/api/contacts/:id', function(req, res) {
  // delete the contact and update the JSON file
});
```

Implementing the user interface using React and Redux

We will now implement the set of features on the client side using React and Redux. First, we will create the folder structure required for writing the code to manage the state and view. This will help us in organising our code better. We will create the folders *store*, *reducers*, *actions*, *api*, *components* under the *src* folder.

Configuring the store

Create a file *configureStore.js* in the folder *store*. Initialise store using the *createStore* API of Redux. We need to provide the *rootReducer*, initial state (if any) and then a store enhancer. Since we will use *redux-thunk*, to implement asynchronous actions, the last parameter will be the *thunk* middleware.

```
createStore( rootReducer, initialState,
  applyMiddleware(thunk) );
```

Implementing the reducers

Create a file *contactReducer.js* under the folder *reducers*. When the store dispatches an action, it passes to the *rootReducer*, the state maintained in the store and the action. While creating the *rootReducer*, we have combined all the individual reducers of the application using the *combineReducers* API. In our current application, we have only one reducer, i.e., for contacts, which we will create shortly. Although we have only one reducer, it will be useful to use the *combineReducers* API because we can extend it to add more reducers as our application expands in the future.